ด้านล่างคือ **ข้อเสนอการแยกเล่ม** จากไฟล์ “โครงสร้าง หลัก” โดยใช้หลัก “1 เล่ม = 1 กลุ่มองค์ความรู้หลัก” และให้ทุกเล่มใช้โครงสร้างหลักเดียวกัน ได้แก่
เรื่อง → วัตถุประสงค์ → เป้าหมาย → ข้อดี → ข้อเสีย → ข้อควรระวัง → ตัวอย่าง → การประยุกต์ใช้ → สรุป
ทั้งในระดับเล่มและระดับบท พร้อมภาคผนวก/แม่แบบท้ายเล่ม

ชุดคู่มือที่แนะนำ: 7 เล่มหลัก + 1 ภาคผนวกกลาง (ถ้าต้องการ)

เล่ม	ชื่อเล่ม	กลุ่มผู้อ่าน	เนื้อหาหลัก	ผลลัพธ์
1	ปฐมบทสถาปัตยกรรมซอฟต์แวร์	Architect, Dev, Tech Lead	พื้นฐานสถาปัตยกรรม, styles/patterns, diagram, architecture-to-code, modularity, testing	เข้าใจภาพรวมและภาษากลางของสถาปัตยกรรม
2	Microservices Architecture	Architect, Dev, DevOps	services, boundary, API, data ownership, communication, deployment, scaling, monitoring, risk mitigation	ออกแบบและประเมินไมโครเซอร์วิสได้
3	Domain-Driven Design และสถาปัตยกรรมที่สะท้อนโดเมน	Architect, Domain Expert, Dev	DDD building blocks, bounded context, context mapping, DDD + Hexagonal/Onion/Clean/CQRS	แยกโดเมนและสะท้อนลงโค้ดได้
4	Clean Architecture และการแยกชั้น	Dev, Architect, Tester	Entities, Use Cases, Interface Adapters, Frameworks & Drivers, Dependency Rule, testing, เปลี่ยนเทคโนโลยี	ออกแบบโค้ดที่แยก business logic ได้
5	การบริหารความเสี่ยงโครงการ	PM, Architect, Tech Lead	ระบุ/ประเมิน/จัดลำดับความเสี่ยง, มาตรการ, ผู้รับผิดชอบ, KPI, แผนสำรอง, Risk Register	จัดทำ Risk Register ได้
6	การวางแผน POC	PM, Architect, Tech Lead	คำถาม, สมมติฐาน, ขอบเขต, ทรัพยากร, เวลา, ตัวชี้วัด, Go/No-Go, สรุปผล	วางแผนและตัดสินใจ POC ได้
7	การบริหารโครงการ	PM, Tech Lead	Scope, Stakeholders, WBS, Timeline, Budget, Resources, Risk, Communication, Quality, KPI, ปิดโครงการ, SMART, บทเรียน	บริหารโครงการได้ครบวงจร
8*	ภาคผนวกและแม่แบบกลาง	ทุกกลุ่ม	Glossary, templates, checklist, diagram notation, case study, cross-reference	ใช้งานร่วมกันทุกเล่ม

* เล่ม 8 เป็นเล่มเสริม ไม่บังคับ

รายละเอียดการแยกเนื้อหาแต่ละเล่ม

เล่ม 1: ปฐมบทสถาปัตยกรรมซอฟต์แวร์

บทที่แนะนำ

1. The Software Architecture Chronicles
2. Software Architecture Premises
3. Architectural Styles vs. Architectural Patterns vs. Design Patterns
4. Architectural Patterns and Styles
5. Monolithic Architecture
6. Component Based Architecture
7. PresentationDomainDataLayering
8. Layers, Hexagons, Features and Components
9. Architectural Drivers: Building Blocks for Decision Making
10. Documenting Software Architecture
11. Simple Sketches for Diagramming your Software Architecture
12. Software Architecture vs Code
13. Mapping Software Architecture to Code
14. An Architecturally-Evident Coding Style
15. Components vs Classes
16. Modularity and Testability
17. Package by Component and Architecturally-Aligned Testing
18. Schools of Package Architecture – An Illustration
19. Aligning Software Architecture and Code
20. Case Study: techtribes.je – Context

หมายเหตุ: “Reflecting architecture and domain in code” **และ** “More than concentric layers” **ใส่เป็นบทนำเชื่อมไป** เล่ม 3–4

เล่ม 2: Microservices Architecture

บทที่แนะนำ

1. **ภาพรวม** Microservices Architecture
2. **บริการย่อย** (Services)

3. **ขอบเขตแต่ละบริการ**
4. API
5. Data Ownership
6. **การสื่อสารระหว่างบริการ**
7. Deployment
8. Scaling
9. Monitoring
10. **ข้อดี/ข้อเสียเมื่อเทียบกับ** Monolithic
11. **แนวทางลดความเสี่ยง**
12. **ตัวอย่างการออกแบบ**
13. **การประยุกต์ใช้และสรุป**

แม่แบบท้ายเล่ม: Service Boundary Canvas, API Contract Checklist, Data Ownership Matrix

---

เล่ม 3: Domain-Driven Design และสถาปัตยกรรมที่สะท้อนโดเมน

บทที่แนะนำ

1. DDD, Hexagonal, Onion, Clean, CQRS: How I Put It All Together
2. Domain / Subdomains
3. Bounded Contexts
4. Ubiquitous Language
5. Entities
6. Value Objects
7. Aggregates
8. Repositories
9. Domain Events
10. Context Mapping
11. **โครงสร้างโค้ดที่สะท้อนโดเมน**
12. Reflecting Architecture and Domain in Code
13. More than Concentric Layers
14. **ตัวอย่างและกรณีศึกษา**
15. **การประยุกต์ใช้และสรุป**

หมายเหตุ: Clean Architecture เชิงลึกอยู่ในเล่ม 4 เพื่อไม่ให้เล่ม 3 ยาวเกินไป

เล่ม 4: Clean Architecture และการแยกชั้น

บทที่แนะนำ

1. ภาพรวม Clean Architecture
2. ชั้น Entities
3. ชั้น Use Cases
4. ชั้น Interface Adapters
5. ชั้น Frameworks & Drivers
6. Dependency Rule
7. การแยก Business Logic ออกจาก Database / Framework / UI
8. วิธีทดสอบ
9. วิธีเปลี่ยนเทคโนโลยีโดยกระทบน้อยที่สุด
10. ข้อดี/ข้อเสีย
11. ตัวอย่างโค้ดและการประยุกต์ใช้
12. สรุป

แม่แบบท้ายเล่ม: Layer Dependency Checklist, Use Case Test Template

---

เล่ม 5: การบริหารความเสี่ยงโครงการ

โครงสร้างบท

1. ระบุความเสี่ยง
2. ประเมินโอกาสเกิด
3. ประเมินผลกระทบ
4. จัดลำดับความเสี่ยง
5. มาตรการลด/ป้องกัน/โอน/ยอมรับ
6. ผู้รับผิดชอบ
7. KPI ติดตาม
8. แผนสำรอง
9. ตัวอย่าง Risk Register
10. การประยุกต์ใช้และสรุป

ตาราง Risk Register ที่แนะนำ

`Risk ID | ความเสี่ยง | สาเหตุ | หมวด | โอกาสเกิด | ผลกระทบ | คะแนน | ลำดับ | มาตรการ | ประเภทมาตรการ | ผู้รับผิดชอบ | KPI | แผนสำรอง | สถานะ`

เล่ม 6: การวางแผน POC

โครงสร้างบท

1. คำถามที่ต้องพิสูจน์

2. **สมมติฐาน**
3. **ขอบเขต**
4. **ทรัพยากร**
5. **ระยะเวลา**
6. **ตัวชี้วัดความสำเร็จ**
7. เกณฑ์ Go / No-Go
8. **สรุปผลและข้อเสนอแนะ**
9. ตัวอย่าง POC Canvas
10. **การประยุกต์ใช้และสรุป**

แม่แบบท้ายเล่ม: POC Canvas 8 ช่องตามโครงสร้างข้างต้น

เล่ม 7: การบริหารโครงการ

บทที่แนะนำ

1. Scope
2. Stakeholders
3. WBS
4. Timeline
5. Budget
6. Resources
7. Risk — เชื่อมเล่ม 5
8. Communication
9. Quality
10. KPI
11. **การปิดโครงการ**
12. **การใช้หลัก** SMART
13. **บทเรียนที่ได้**
14. **ตัวอย่างและสรุป**

แม่แบบท้ายเล่ม: Project Charter, WBS Template, Stakeholder Matrix, Communication Plan, Lessons Learned Template

---

หลักการแยกเล่มที่ควรยึด

1. **แต่ละเล่มมี theme เดียว** ไม่ปนพื้นฐานสถาปัตยกรรมกับ项目管理 มากเกินไป
2. **ใช้โครงสร้างหลักเดียวกันทุกเล่ม** เพื่อให้อ่านและอ้างอิงง่าย

3. **ทุกบทมีตัวอย่าง + ข้อควรระวัง + การประยุกต์ใช้**
4. **เชื่อมโยงข้ามเล่มด้วย cross-reference** เช่น Risk ในเล่ม 7 ชี้ไปเล่ม 5, Clean Architecture ในเล่ม 3 ชี้ไปเล่ม 4
5. **มี Glossary กลาง** และแม่แบบกลางในเล่ม 8 หรือท้ายแต่ละเล่ม
6. **ลำดับการอ่านแนะนำ:** เล่ม 1 → 2 หรือ 3 → 4 → 5 → 6 → 7

ถ้าต้องการ ผมสามารถจัดทำ **สารบัญแยกเล่มแบบละเอียด** หรือ **ไฟล์ Markdown แยกทีละเล่ม** ตามโครงสร้างนี้ให้ต่อได้